Paul Parisot

Software Engineer - Low-Latency & Systems Programming

Github: paulogarithm — paul.parisot@epitech.eu - +33 6 42 36 01 57

About

Systems software engineer deeply focused on hardware-level optimization, low-latency architectures, and memory deterministic designs.
Extensive practical exploration of thread synchronization, lock-free concepts, and asynchronous network multiplexing.
Driven by predictability, cache-friendly data layouts, and squeezing maximal performance out of every CPU cycle under Linux.

Skills

Core Automation: C, C++ (Modern standards), Python, Bash, SQL, Go, Assembly
Concurrency & Memory: Atomic primitives (`_Atomic` / `std::atomic`), Multithreading, Custom Allocators, Cache-line alignment
Networking & Sockets: Network Multiplexing (`select`), Asynchronous I/O (`Boost.Asio`), TCP/UDP Sockets
Software Engineering: Linux Kernel primitives, Performance Profiling, Git Toolchain, Automated CI/CD pipelines

Professional Experience

Gno.land - Core Software Engineer (Distributed Systems)

- Designed core modules in a distributed ecosystem, enforcing execution determinism and strict data consistency.
- Diagnosed and resolved **6 critical multi-threaded race conditions** and runtime memory corruptions.
- Implemented high-throughput testing suites under strict peer-review policies, delivering **10k lines of production code**.

Faurecia / Forvia - Software Engineer (High-Performance C++)

- Engineered high-frequency C++ network modules for real-time embedded telemetry, optimizing raw packet processing.
- Implemented deterministic routing algorithms for navigation systems showcased at **CES 2024**.
- Utilized automated testing paradigms to validate throughput benchmarks and guarantee minimal processing jitters.

Epitech / IONIS - Systems Programming Instructor

- Mentored **400+ students** in advanced systems architecture, low-level memory management, and C/C++ memory layouts.
- Enforced strict evaluation policies regarding race-condition identification, thread safety, and resource leaks.

Low-Latency & Systems Projects

High-Throughput Network Engine (Zappy)

Architected a pure C single-threaded network server utilizing system-level `select` multiplexing for non-blocking I/O.
Achieved a **100% leak-free** memory footprint under rigorous validation, engineered entirely as a solo developer.
Stress-tested with multi-gigabyte data ingestion streams (`/dev/random`) to validate protocol parsing stability and eliminate buffer fragmentation.

Skills: Pure C, Network Multiplexing, Non-blocking Sockets, Linux I/O, Stress Testing, Memory Sanitizers

rodox – Low-Latency Systems Engine

Developed a Linux C++ engine utilizing strict data-oriented structures (ECS layout) to optimize L1/L3 CPU cache locality.
Designed custom **static pool allocators** to completely bypass runtime heap allocations (`malloc`/`new`) during hot execution paths.
Integrated `Boost.Asio` for asynchronous network handling alongside hardware-accelerated SIMD instruction sets.

Skills: C++, Boost.Asio, Custom Allocators, Cache-Line Optimization, SIMD Vectorization, Modern C++

Theoretical Research & Advanced Hacks

- **Computer Science Theory**: In-depth preparation for advanced theoretical competitions, specializing in concurrency formalisms, automata, and operational semantics.
- **Runtime Assembly Injection (cclass)**: Engineered a custom engine using Linux `mmap` pages to hot-patch compiled assembly stubs, bypassing traditional dispatch tables overhead.

Education

Epitech, Paris 2022 – 2027
Master's Degree in Information Technology (Level-7 RNCP Certification) – Specialization: Systems Architecture

McGill University, Montréal 2025 – 2026
Certificate in Management – Technical Leadership & Strategy